Universidad Nacional Autónoma de México
Facultad de Ingeniería
Inteligencia Artificial

Tarea #2 : Selección de algoritmos y resolución de problemas

Alumno: Vera Morales Juan Pablo
Grupo: 10
Profesor : Romero de la Cruz Alejandro

Introducción

El siguiente reporte comprende la implementación y análisis de los algoritmos vistos en clase (A*, búsqueda local, CSP / Backtracking / AC-3, etc.) en la resolución de distintos problemas, incluyendo la justificación de la selección del algoritmo utilizado para cada uno y los resultados reportados para cada problema.

Herramientas

Para la realización de la tarea se utilizó el lenguaje Python y el editor de código Visual Studio Code.

Desarrollo

Problema de la mochila 10 y 100.

1) Modelado

Definiremos cada uno de los aspectos del modelado, mismo que seguiremos para el de problemas posteriores, posteriormente su configuración para el problema actual.

Variables/Estado

Las variables se refieren a todos aquellos elementos individuales que pueden tomar un valor, el estado en cambio es la configuración completa de todas las variables en un momento dado, para representar el problema de la mochila utilizaremos un arreglo unidimensional donde cada espacio dentro del arreglo será una variable y el conjunto de elementos que conforman ese arreglo el estado.

Dominio

El dominio es el conjunto de valores que puede tomar cada variable, para este problema nuestro dominio sera {0,1} , un 0 representa la ausencia del elemento en la mochila mientras que 1 representa que se encuentra dentro.

Acciones/Vecindad

Estas definen como nuestro estado es transformado para explorar el espacio de soluciones, las acciones se representan en los niveles de un arbol, la accion es decidir sobre el siguiente objeto disponible, en cambio el concepto de vecindad utiliza a los “vecinos” que son aquellos espacios que difieren del estado actual por una modificacion minima.

Para cada nivel del arbol “i” , siendo “i” el indice del objeto del archivo .csv , el agente tendra dos caminos:

a) Incluir el objeto “i”: Suma el peso y el valor del objeto al estado actual siempre y cuando no exceda la capacidad maxima definida en capacidades_mochila.txt
b) No incluir el objeto i: Se pasa al siguiente objeto sin modificar peso ni valor acumulado.

Se transiciona al ejecutar una acción lo cual lleva de un estado de decision k a un estado k + 1 construyendo la solucion paso a paso hasta haber decidido sobre los 10 objetos.

Restricciones

Las restricciones son las reglas o condiciones que debe cumplir la solución para ser considerada valida, en el problema de la mochila solo existe 1:

- El peso total de los objetos seleccionados no debe exceder la capacidad maxima pemitida.

Funcion objetivo / Costo y / o Heuristica

La funcion objetivo hace referencia al proposito principal del problema y representa la metrica que se quiere maximizar o minimizar (dependiendo del enfoque del problema) para considerar que una solución es mejor, el costo es una medida que cuantifica la penalizacion o gasto asociado a una configuración especifica o a la transición entre estados, finalmente la heuristica es una funcion de estimacion que no granantiza la solucion pero ayuda a acotar el espacio de soluciones.

Funcion Objetivo: Maximizar la suma de los varones de los objetos incluidos.

Heuristica: Relajacion Lineal, utilizada ampliamente en A* puesto que es admisible , es decir que nunca sobreestima el valor real si se plantea como minimizacion de “valor perdido” o se busca un limite superior.

2) Algoritmos elegidos y justificación

Algoritmo elegido

Para solucionar este problema se utilizo el algoritmo A* como ya se ha mencionado y la heuristica Relajación Lineal,

Justificación

La combinación de A* junto con la heuristica utilizada conforman una solución optima, en un inicio el problema se expande hasta 2^{10} combinaciones (1024) posibles en una busqueda de fuerza bruta pero nuestro algoritmo en conjunto reduce el esfuerzo mediante la poda de nodos no prometedores, esto se hace mediante tres factores, primero relajacion lineal conoce el potencial maximo de una rama antes de investigarla, si el algoritmo ya encontro una mochila con un potencial de 250 no investiga una cuyo maximo seria 240 por lo que esta rama se descarta, segundo la exploración por ratio que explora objetos muy valiosos y ligeros encontrando las mejores soluciones antes, finalmente al añadir la restriccion del peso A* no genera nodos con valores de "peso" que excedan el limite, este algoritmo ignora subarboles que incumplen restricciones o son matematicamente inferiores.

3) Propiedades teoricas

- **Completitud:** El algoritmo es completo, dado que el espacio de estados es finito , un arbol de profundidad 10 con factor de ramificación 2, y el algoritmo explora todos los nodos que necesita de la frontera de busqueda siempre encontrara una solucion mientras que exista una combinación que cumpla con la restricción del peso.

- **Optimalidad:** El algoritmo es optimo debido a que la heuristica que utilizamos es admisible ya que nunca subestima el valor maximo que se puede alcanzar desde un nodo dado gracias al uso del ratio, bajo esta condicion A* encuentra la mejor solución posible.}

- **Complejidad:**
    - **Tiempo :** En el peor de los casos $O(b^d)$ donde b es igual al numero de valores que puede tomar una variable, en este caso 2, y d es igual al numero de objetos, en este caso 10, aunque por la heuristica y poda utilizada el numero de expansiones tiende a ser mucho menor.
    - **Memoria:** Igualmente es $O(b^d)$ puesto que el algoritmo necesita mantener en la lista abrir todos los estados generados aun no expandidos que pueden ser todos en el peor de los casos.

4) Resultados

Mochila 10 Objetos

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea
2\mochila_10_a_estrella.py"
Valor acumulado: 270
Estado solucion: [1, 1, 1, 1, 1, 0, 1, 1, 0, 0]
Objetos en la mochila: [1, 2, 3, 4, 5, 7, 8]
Peso maximo: 38
Peso utilizado: 36
Nodos expandidos: 11
Tiempo de ejecuci►n: 0.000051s

[Done] exited with code=0 in 0.702 seconds
```

Mochila 100 Objetos

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea
2\mochila_100_a_estrella.py"
Valor acumulado: 8373
Estado solucion: [1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0,
1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0,
0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1,
0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0]
Objetos en la mochila: [1, 4, 9, 11, 13, 15, 16, 17, 18, 19, 23, 24, 28, 30, 31, 32,
33, 34, 36, 39, 41, 42, 45, 47, 49, 52, 53, 57, 58, 60, 61, 62, 63, 64, 65, 66, 67,
68, 69, 70, 74, 76, 77, 78, 80, 81, 82, 83, 84, 86, 87, 88, 89, 90, 91, 92, 94, 95,
97, 98]
Peso maximo: 1097
Peso utilizado: 1096
Nodos expandidos: 3963
Tiempo de ejecuci►n: 0.065035s
```

5) Reflexion Corta

Lo mas complicado fue el ciclo while de la cola “abrir” todavia me cuesta entender como funciona la exploración desde las colas , entiendo que la reorganización realizada por la heuristica hace que lo mas prometedor este al inicio y sea lo primero que investigamos sin embargo me sigue costando visualizarlo, no cambiaria mucho del codigo puesto que en 10 objetos encontrar la solucion en 11 nodos y en 100 en 3963 se me hacen numeros inmejorables.

N-Reinas (8 y 100)

1) Modelado

Variables/Estado

Utilizaremos un vector de tamaño n (8 y 100 respectivamente para cada solucion) cada variable Xi representara a una renina en la columna i, el valor de la variable representa la fila en la que se encuentra.
El estado sera la configuracion completa del tablero donde cada columna tiene exactamente una reina.

Dominio

El dominio es el conjunto de numeros {0,1, … , n - 1} que representan las filas posibles de la cuadricula del tablero.

Acciones/Vecindad

El algoritmo que utilizaremos sera tabu para el cual se utilizan vecindades, la vecindad se define moviendo a una reina seleccionada que se encuentre en conflicto con otra a una fila distinta dentro de su misma columna.

Restricciones

Las restricciones para el problema N-Reinas son 2:

- No puede habar dos reinas en una misma fila $Xi \neq Xj$
- No puede haber dos reinas en la misma diagonal ($|Xi - Xj| \neq |i - j|$ para todo $i \neq j$)

Funcion objetivo / Costo y / o Heuristica

Costo: El numero total de pares de reinas que se atacan entre si.

2) Algoritmos elegidos y justificación

Algoritmo elegido

Para solucionar este problema se escogio Tabu de la lista de algoritmos de busqueda local.

Justificación

Tabu es una metaheurística de búsqueda local muy competente para problemas de gran escala como las 100-reinas a diferencia de hill climbing que puede quedar atrapado en óptimos locales la búsqueda tabu utiliza una lista para prohibir temporalmente movimientos que nos

regresarían a estados ya visitados esto obliga al algoritmo a explorar nuevas regiones del espacio de búsqueda y a "escapar" de mesetas de conflictos, lo cual es vital para encontrar una solución de costo cero en tableros grandes de manera eficiente.

3) Propiedades teoricas

- **Completitud:** Esta implementacion no es completa, aunque esta diseñada para escapar de optimos locales mediante la lista tabu y el criterio de aspiracion el algoritmo depende de que la solucion se encuentre dentro del limite de maximas iteraciones, si el espacio de busqueda es muy complejo o tenure (K) no esta bien ajustado el algoritmo puede terminar sin encontrar el estado con 0 conflictos.

- **Optimalidad:** El algoritmo no garantiza la optimalidad , en el contexto del problema n-reinas la unica solucion optima es 0 conflictos, el algoritmo busca esta solucion pero si alcanza el limite antes de encontrarla retornara el “best” que podria tener una cantidad de ataques mayor a 0.

- **Complejidad:**
  - **Tiempo :** La complejidad por iteración puede alcanzar $O(n^3)$ debido a la generacion de vecinos, cada uno con copia de la lista list(s) lo que ya cuesta O(n).
  - **Memoria:** La complejidad en memoria tambien alcanza los $O(n^3)$ ya que la función obtener vecinos almacena simultaneamente en la lista vecinos todos los estados posibles de vecino, como el consumo de memoria ya es n^2 esto añade otra lista tamaño n, la lista tabu tiene complejidad menor a $O(k)$ donde k es el tenure.

4) Resultados

8 Reinas

1. **Tabu encontro una solución optima**

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea 2\n-reinas-8-tabu.py"
Conflictos finales (Costo): 0
Estado soluci►n (Filas): [5, 3, 1, 7, 4, 6, 0, 2]
Tama►o del tablero (N): 8
Limite de lista tabu (k): 5
Iteraciones realizadas: 6
Tiempo de ejecuci►n: 0.000797s
```

2. **Tabu no encontro una solucion optima y retorno su “Best”**

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea 2\n-reinas-8-tabu.py"
Conflictos finales (Costo): 1
Estado soluci►n (Filas): [4, 6, 0, 3, 5, 7, 2, 6]
Tama►o del tablero (N): 8
Limite de lista tabu (k): 5
Iteraciones realizadas: 200
Tiempo de ejecuci►n: 0.037959s
```

100 Reinas

1. **Tabu encontro una solución optima.**

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea 2\n-reinas-100-tabu"
Conflictos finales (Costo): 0
Estado soluci►n (Filas): [90, 39, 85, 22, 13, 21, 7, 9, 28, 70, 14, 59, 94, 89, 31, 84, 73, 98, 46, 58, 51,
17, 78, 65, 87, 43, 50, 40, 6, 4, 33, 72, 23, 68, 41, 25, 3, 19, 93, 15, 11, 56, 82, 38, 74, 53, 79, 76, 48,
8, 64, 32, 86, 5, 24, 77, 61, 36, 1, 80, 26, 0, 45, 99, 44, 63, 60, 2, 37, 66, 97, 20, 49, 96, 30, 81, 18,
42, 52, 16, 92, 67, 10, 27, 69, 95, 75, 71, 34, 47, 62, 29, 54, 57, 55, 83, 88, 12, 91, 35]
Tama►o del tablero (N): 100
Limite de lista tabu (k): 15
Iteraciones realizadas: 57
Tiempo de ejecuci►n: 11.949713s
```

2. **Tabu no encontro una solucion optima y retorno su “Best”, para n mas grandes el tiempo de ejecución puede ser exagerado.**

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea 2\n-reinas-100-tabu"
Conflictos finales (Costo): 1
Estado soluci►n (Filas): [22, 82, 73, 53, 12, 59, 79, 46, 36, 4, 35, 40, 70, 25, 38, 19, 61, 77, 62, 67, 76,
88, 13, 17, 9, 44, 15, 43, 96, 28, 6, 1, 63, 14, 87, 69, 74, 0, 85, 37, 99, 50, 29, 49, 2, 71, 56, 18, 52,
86, 64, 68, 75, 58, 95, 90, 57, 41, 54, 92, 33, 47, 10, 27, 24, 72, 21, 32, 65, 48, 83, 7, 31, 11, 89, 93, 3,
55, 98, 26, 42, 81, 84, 94, 5, 51, 39, 20, 60, 8, 16, 66, 23, 45, 97, 78, 30, 34, 80, 91]
Tama►o del tablero (N): 100
Limite de lista tabu (k): 15
Iteraciones realizadas: 2500
Tiempo de ejecuci►n: 854.582873s

[Done] exited with code=0 in 854.691 seconds
```

5) Reflexion Corta

Entendiendo que son problemas y aproximaciones distintas pero me parecio muy dramatica la diferencia en performance de A* comparada con busqueda tabu, algoritmo que me fue mas complicado implementar e incluso tuve que buscar la forma de simplificar en tiempo la funcion calcular_ataques porque de por si el codigo se estaba haciendo muy tardado, aunque puede funcionar para problemas chicos cuando crece como en el caso de la ejecución mostrada para 100 reinas el algoritmo puede ser muy ineficiente.

Sudoku 9 x 9

1) Modelado

Variables/Estado

Las variables son cada una de las celdas vacias dentro de la cuadricula de 81 celdas del sudoku 9 x 9 , un estado seria una asignación parcial o completa de numeros a estas celdas, en el archivo del que se tomaran los datos 0 representa una celda vacia.

Dominio

El dominio para cada celda vacia es el conjunto de numeros enteros: {1,2,3,4,5,6,7,8,9}.

Acciones/Vecindad

El algoritmo que se utilizara sera backtracking el cual utiliza el enfoque de acciones, una accion se definira como asignar un numero del dominio a una celda vacia que no rompa con las restricciones inmediatas.

Restricciones

Para que una solucion sea considerada optima se necesitan 3 cosas:

- Cada numero del 1 al 9 debe aparecer exactamente una vez por fila.
- Cada numero del 1 al 9 debe aparecer exactamente una vez por columna.
- Cada numero del 1 al 9 debe aparecer exactamente una vez en cada sub region 3x3.

Funcion objetivo / Costo y / o Heuristica

Objetivo: Encontrar una asignacion completa donde todas las varibles tengan un valor y no se rompa ninguna restriccion.

2) Algoritmos elegidos y justificación

Algoritmo elegido

El algoritmo seleccionado fue Backtracking

Justificación

Se selecciono backtracking porque el problema sudoku 9x9 tiene restricciones explicitas para fila , columna y subregion que permiten podar el arbol de busqueda de forma inmediata al detectar asignaciones invalidas, aunque el espacio de busqueda teorico sea 9^{81} para un tablero 9x9 la busqueda realizada es muy eficiente y encuentra la solucion en poco tiempo, este

algoritmo garantiza una solucion valida (si existe) sin la necesidad de implementar mecanismos mas complejos como arcos o ordenamiento.

3) Propiedades teoricas

- **Completitud:** El algoritmo es completo, dado que el numero de celdas y dominio de cada uno son finitos, el algoritmo explorara sistematicamente cada combinacion posible hasta dar con la solución o determinar que no existe bajo las restricciones dadas.

- **Optimalidad:** El algoritmo cumple con optimalidad porque proporciona soluciones que cumplen con las restricciones de encontra una solucion costo 0, es decir sin conflictos, si esta existe.

- **Complejidad:**
    - **Tiempo :** En el peor caso es $O(d^n)$ donde d es el tamaño del dominio, en este caso 9, n el numero de variables, es decir los cuadros del sudoku que para este tamaño son 81, sin embargo las restricciones del sudoku realizan una poda que minimiza el numero de nodos expandidos.
    - **Memoria:** Es $O(n)$ donde n igualmente es el numero de variables, el algoritmo solo guarda el estado actual del tablero y la pila recursiva cuyo tamaño se limita a el numero de celdas vacias que tiene que llenar.

4) Resultados

Solución para el sudoku propuesto.

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea 2\sudoku_9x9_backtracking.py"
Estado soluci►n:
[5, 3, 4, 6, 7, 8, 9, 1, 2]
[6, 7, 2, 1, 9, 5, 3, 4, 8]
[1, 9, 8, 3, 4, 2, 5, 6, 7]
[8, 5, 9, 7, 6, 1, 4, 2, 3]
[4, 2, 6, 8, 5, 3, 7, 9, 1]
[7, 1, 3, 9, 2, 4, 8, 5, 6]
[9, 6, 1, 5, 3, 7, 2, 8, 4]
[2, 8, 7, 4, 1, 9, 6, 3, 5]
[3, 4, 5, 2, 8, 6, 1, 7, 9]
Tama►o del tablero: 9x9
Tiempo de ejecuci►n: 0.029659s
Iteraciones totales: 37652
```

Demostracion, cambiando un valor del txt para hacer el sudoku invalido:

```
0 representa una celda vacía
5 3 3 0 7 0 0 0 0
6 0 0 1 9 5 0 0 0
0 9 8 0 0 0 0 6 0
8 0 0 0 6 0 0 0 3
4 0 0 8 0 3 0 0 1
7 0 0 0 2 0 0 0 6
0 6 0 0 0 0 2 8 0
0 0 0 4 1 9 0 0 5
0 0 0 0 8 0 0 7 9
```

En la linea 2 se añadio otro 3 a la subregion 3x3 haciendo de este sudoku imposible de solucionar para lo que nuestro algoritmo retorna:

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea 2\sudoku_9x9_backtracking.py"
No se encontro una soluci⍰n para el tablero propuesto
```

5) Reflexion Corta

Para este punto habiendo solucionado dos de los problemas mediante acciones y uno con vecindades identifico, al menos hasta ahora, que las soluciones por acciones parecen tender mas a cumplir con completitud, optimalidad y menores complejidades en tiempo y memoria, quiza este problema podria optimizarse aun mas utilizando alguna heuristica mas agresiva como MRV empezando por aquellas celdas que se encuentran mas restringidas.

Sudoku 16 x 16

1) Modelado

Variables/Estado

Las variables son cada una de las celdas vacias dentro de la cuadricula de 256 celdas del sudoku 16 x 16 , un estado seria una asignación parcial o completa de numeros a estas celdas, en el archivo del que se tomaran los datos 0 representa una celda vacia.

Dominio

El dominio para cada celda vacia es el conjunto de numeros enteros y letras: {1,2,3,4,5,6,7,8,9,A,B,C,D,E,F}.

Acciones/Vecindad

El algoritmo que se utilizara sera backtracking el cual utiliza el enfoque de acciones pero esta vez debido a la mayor complejidad utilizaremos consistencia de arcos, MRV y LRV, esto tambien nos ayudara a ver las mejoras de utilizar heuristicas mas complejas como se propuso en la reflexión de sudoku 9x9 , una accion se definira como asignar un simbolo del dominio a una celda vacia que no rompa con las restricciones inmediatas.

Restricciones

Para que una solucion sea considerada optima se necesitan 3 cosas:

- Cada simbolo del 1 a la F debe aparecer exactamente una vez por fila.
- Cada numero del 1 a la F debe aparecer exactamente una vez por columna.
- Cada numero del 1 a la F debe aparecer exactamente una vez en cada sub region 4x4.

Funcion objetivo / Costo y / o Heuristica

Objetivo: Encontrar un estado completo y valido donde no se incumpla ninguna restricción para los 256 campos del sudoku 16 x 16.

Heuristicas:

MRV(Variable mas restringida), selecciona la celda vacia que tenga el menor numero de valores legales en su dominio acelerando la deteccion de fallos.
LVR(Valor menos restrictivo= Selecciona el valor que descarte la menor cantidad de opciones para las variables vecinas en el grafo de restricciones.
AC-3: Mantener la consistencia de arcos eliminando valores de los dominios de las variables vecinas cada vez aue se realiza una asignacion lo que reduce el arbol de busqueda.

2) Algoritmos elegidos y justificación

Algoritmo elegido

El algoritmo elegido fue backtracking pero ahora utilizadno las heuristicas mrv, lrv, ac-3.

Justificación

Debido al crecimiento en la complejidad del calculo no considere optimo seguir con el backtracking simple utilizado en 9x9 ya que este es un problema mucho mas robusto, al principio crei que bastaria con lcv y mrv pero se tuvo que implementar un forward checking debido a lo extenso de el arbol de busqueda lo que me hace pensar que mi consideracion en la mejora a la potencia de calculo que iba a necesitar fue correcta.

3) Propiedades teoricas

- **Completitud:** El algoritmo es completo, debido a que utiliza una búsqueda con retroceso (backtracking) sobre un espacio de estados finito (256 variables con dominios de 16 símbolos), el algoritmo explorará sistemáticamente todas las asignaciones posibles hasta encontrar una que satisfaga todas las restricciones o determinar que no existe solución

- **Optimalidad:** El algoritmo es optimo puesto que granatiza encontrar una solucion con costo cero siempre y cuando esta exista.

- **Complejidad:**
  - **Tiempo :** En el peor caso es $O(d^n)$, donde d=16 símbolos del 0 al F, y n=256 total de celdas, sin embargo la poda del ac-3 y mrv reducen el tiempo de ejecucion.
  - **Memoria:** Es de O(n), siendo n=256, el algoritmo solo mantiene en memoria una instancia del tablero que se modifica y la pila de recursión

4) Resultados

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea 2\sudoku_16x16_heuristicas.py"
Iteraci►n: 0 | Celdas asignadas: 0
Iteraci►n: 10000 | Celdas asignadas: 6
Iteraci►n: 20000 | Celdas asignadas: 7
0 9 2 B 4 6 7 5 8 1 3 A C D E F
6 5 4 7 8 B A D C E 9 F 2 0 1 3
8 1 A F C 0 E 3 B D 2 4 5 9 6 7
C 3 E D 2 1 9 F 0 5 6 7 8 A 4 B
2 C 3 4 6 7 F 0 1 A B 8 D 5 9 E
D F 5 8 9 A B C 4 7 E 0 1 2 3 6
9 A B 1 D 8 5 E 2 3 C 6 0 F 7 4
E 6 7 0 1 2 3 4 5 9 F D B 8 A C
F E 8 C 5 D 6 9 A 2 7 3 4 B 0 1
B 7 6 9 A 4 C 8 D F 0 1 E 3 2 5
A D 0 3 B F 2 1 E C 4 5 6 7 8 9
4 2 1 5 E 3 0 7 6 B 8 9 A C F D
5 4 C 6 7 9 8 A 3 0 D E F 1 B 2
3 8 9 E 0 C D B F 6 1 2 7 4 5 A
7 0 D A F E 1 2 9 4 5 B 3 6 C 8
1 B F 2 3 5 4 6 7 8 A C 9 E D 0
Iteraciones: 22451
Tiempo total: 35.519520s

[Done] exited with code=0 in 35.715 seconds
```

5) Reflexion Corta

Este problema me mostro que no se puede subestimar que tanto se necesita para solucionar un problema, aunque instintivamente pense que aumentar a 16x16 no supondria una tarea tan grande y con las heuristicas mencionadas al principio bastaria, despues de depurar el codigo y notar que el algoritmo constantemente entraba en zonas muertas note que se deben tener mas consideraciones a la hora de atacar un problema.

Coloreo de mapas

1) Modelado

Variables/Estado

Se utilizara un vector de tamaño "n" donde n sera la cantidad de paises y cada posición representara un pais y su color, el estado seria la asignacion completa de colores a todos los paises del mapa.

Dominio

El dominio esta conformado por el conjunto de colores posibles, dado que el objetivo es minimizar el tamaño de este empezamos con un dominio pequeño {1, 2 , … , k} y aumentamos de ser necesario.

Acciones/Vecindad

Vecindad: Se define seleccionando un pais al azar y cambiando su color por cualquier otro valor dentro del dominio, la transicion se realiza aceptando vecinos "peores" a cambio de la probabilidad de escapar de optimos locales.

Restricciones

El problema de colore de paises solo tiene una restriccion:

- Dos paises que comparten frontera no pueden tener el mismo color.

Funcion objetivo / Costo y / o Heuristica

Funcion costo: El numero total de conflictos, es decir, de paises que comparten una frontera.

Funcion Objetivo: El algoritmo termina cuando el costo llega a 0.

2) Algoritmos elegidos y justificación

Algoritmo elegido

El algoritmo utilizado para solucionar este problema es recocido simulado.

Justificación

El algoritmo de Recocido Simulado funciona bien en terminos de este problema por el extenso espacio de busqueda (k^{100}) que seria complicado de tratar con algoritmos exhaustivos, esto se debe a su capacidad para escapar de optimos locales mediante la aceptacion de movimientos que aceptan temporalmente el costo basado en la probabilidad controlada por la temperatura, esta implementación explora el espacio de manera eficiente adaptandose bien a las limitaciones binarias.

3) Propiedades teoricas

- **Completitud:** No es estrictamente completa, al ser busqueda local y estocastica no garantiza encontrar una solucion , su exito depende en gran medida de un buen ajuste en temperatura inicial y factor de enfriamiento, que ambos permitan una explioracion suficiente antes de que el sistema se congele o alcance las maixmas iteraciones.

- **Optimalidad:** El algoritmo no garantiza la optimilidad teorica, sin embargo para un k dado busca una solucion con costo 0 lo cual es el optimo para estos problemas.

- **Complejidad:**
  - **Tiempo :** Es O(E * T) Donde T es el numero de iteraciones realizadas y E el numero de aristas del mapa, en cada iteracion calcular_costo recorre todas las conexiones del diccionario para verificar conflictos.
  - **Memoria:** Es O(V + E) Donde V es el numero de paises y E el de adyacencias, se almacena el grafo en el diccionario adj y el estado actual es un diccionario de tamaño V.

4) Resultados

Recocido simulado para n = 5

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea 2\coloreo_mapas_5_paises_recocido_simulado.py"
Estado soluci►n: {'A': 2, 'B': 3, 'C': 2, 'D': 1, 'E': 3}
Dominio (k): 3
Iteraciones realizadas: 23
Tiempo de ejecuci►n: 0.000268s
```

Recocido simulado para n = 100

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\IA 2026-2\Tarea
2\coloreo_mapas_100_paises_recocido_simulado.py"
Estado soluci⌂n: {'P001': 1, 'P002': 3, 'P003': 2, 'P004': 3, 'P005': 1, 'P006': 2, 'P007': 1,
'P008': 3, 'P009': 1, 'P010': 3, 'P011': 3, 'P012': 2, 'P013': 3, 'P014': 2, 'P015': 3, 'P016': 1,
'P017': 2, 'P018': 1, 'P019': 2, 'P020': 1, 'P021': 1, 'P022': 3, 'P023': 2, 'P024': 3, 'P025': 1,
'P026': 2, 'P027': 3, 'P028': 2, 'P029': 3, 'P030': 2, 'P031': 3, 'P032': 1, 'P033': 3, 'P034': 1,
'P035': 2, 'P036': 3, 'P037': 2, 'P038': 3, 'P039': 2, 'P040': 1, 'P041': 1, 'P042': 3, 'P043': 1,
'P044': 3, 'P045': 1, 'P046': 2, 'P047': 1, 'P048': 2, 'P049': 1, 'P050': 3, 'P051': 3, 'P052': 1,
'P053': 2, 'P054': 1, 'P055': 3, 'P056': 1, 'P057': 2, 'P058': 1, 'P059': 3, 'P060': 1, 'P061': 2,
'P062': 3, 'P063': 1, 'P064': 3, 'P065': 1, 'P066': 3, 'P067': 1, 'P068': 2, 'P069': 1, 'P070': 2,
'P071': 3, 'P072': 1, 'P073': 3, 'P074': 1, 'P075': 2, 'P076': 1, 'P077': 2, 'P078': 1, 'P079': 3,
'P080': 1, 'P081': 1, 'P082': 3, 'P083': 1, 'P084': 2, 'P085': 3, 'P086': 2, 'P087': 1, 'P088': 2,
'P089': 1, 'P090': 2, 'P091': 2, 'P092': 1, 'P093': 2, 'P094': 3, 'P095': 2, 'P096': 3, 'P097': 2,
'P098': 1, 'P099': 2, 'P100': 3}
Dominio (k): 3
Iteraciones realizadas: 1712
Tiempo de ejecuci⌂n: 0.054637s

[Done] exited with code=0 in 0.184 seconds
```

5) Reflexión Corta

Este algoritmo particularmente me mostró que la reflexión que tuve con tabu estaba sesgada por un problema para el que no funcionaba particularmente bien, en problemas como estos, donde una búsqueda exhaustiva hubiera sido más complicada estos algoritmos utilizando probabilidades demostraron ser altamente eficientes aun sin cumplir completitud ni optimalidad.

Cuadro magico grande

1) Modelado

Variables/Estado

Para el algoritmo a utilizar, hillclimbing, las variables son cada una de las celdas con coordenadas Xij , el estado es una matriz de 10 x 10 que contiene los numeros , todos distintos.

Dominio

El dominio son los numeros enteros {1 , 2 , 3 , … , 99 , 100} cada uno se permite una unica vez por estado.

Acciones/Vecindad

La vecindad es el conjunto de todos los cuadros posibles que se dan al realizar un unico cambio en el cuadro actual, las transformaciones del mismo se dan intercambiando cualquier celda Xij de lugar con Xyz donde (i , j) $\neq$ (y , z), elegidas al azar o con algun criterio de error.

Restricciones

Este problema cuenta con 4 restricciones:

- $\sum_{i=1}^{10}$ X i , j = 505 para cada fila de la matriz 10 x 10.
- $\sum_{j=1}^{10}$ X i , j = 505 para cada columna de la matriz 10 x 10.
- $\sum_{i=1}^{10}$ X i , i = 505 y $\sum_{i=1}^{10}$ X i , 11 - i = 505 para las diagonales principales de la matriz 10 x 10.
- $i1, j1, i2, j2 \in \{1, 2, ..., 10\},\ si\ (i1, j1) = (i2, j2),\ entonces\ ai1j1 \neq ai2j2$

Funcion objetivo / Costo y / o Heuristica

Funcion Objetivo: Alcanzar un estado de costo 0 representando un cuadro magico perfecto.

Costo:

$$\sum|Suma_Filai - 505| + \sum|Suma_Columnaj - 505| + \sum|Suma_Diagk - 505|$$

2) Algoritmos elegidos y justificación

Algoritmo elegido

El algoritmo selecciónado (inicialmente) fue Hill Climbing, pero debido a que el algoritmo superaba las maximas iteraciones y no terminaba solucionando el problema (casi siempre oscilando entre costo 1 y 3) se añadio la heruistica de intercambio dirigido, la idea central de esto es que se buscaran filas con fallos “compatibles” para intentar compensar con valores de una que le faltaran a la otra, esto genero otro estancamiento, el algoritmo rapidamente encontraba optimos locales y por ser hill climbing se negaba a tomar vecinos que lo

empeoraran, por lo que además se elimino la heuristica dirigida y se termino añadiendo reinicios aleatorios con un indice de “paciencia”.

Justificación

Este enfoque de búsqueda local se adapta al problema debido al tamaño astronómico del espacio de búsqueda el cual imposibilita el uso de algoritmos de búsqueda exhaustiva, la justificación de usar intercambios en la vecindad es que permite mantener satisfecha en todo momento la restricción de "números del 1 al 100 sin repetir", simplificando el modelado del problema, el uso de reinicios aleatorios activados por “paciencia” es la clave para este problema porque permite al algoritmo escapar de óptimos locales las configuraciones mencionadas anteriormente donde casi todas las sumas dan 505 pero no se puede mejorar más sin deshacer lo avanzado y explorar nuevas regiones del espacio de estad.

3) Propiedades teoricas

- **Completitud:** El algoritmo no es completo, aunque los reinicios aleatorios aumentan la posibilidad de encontrar una solucion no existe una garantia de que encuentre un espacio solucion con costo 0.

- **Optimalidad:** No asegura el encontrar un algoritmo costo 0 , devuelve la mejor solución encontrada.

- **Complejidad:**
    - **Tiempo :** La complejidad por iteracion es $O(n^2)$ donde n = 10 por la funcion calcular_costo que recorre la matriz completa para sumar filas, columnas y diagonales, luego se multiplica por el numero total de iteraciones dando un costo en tiempo de $O(T * n^2)$.
    - **Memoria:** $O(n^2)$ solamente necesitando almacenar en memoria el cuadro actual, el vecino y el “mejor cuadro”.

4) Resultados

Resultados primer codigo, sin busqueda dirigida, sin reinicios aleatorios.

```
Problema: Cuadro M►gico 10x10
Constante M►gica Objetivo: 505
Costo Final (Error acumulado): 3
Iteraciones realizadas: 1000000
Tiempo total: 0.60s

Visualizaci►n del Cuadro (10x10):
--------------------------------------------------
|  17 100  88  42  54  52  59  65   2  26 |
|   6  83   1  75  48  82  53  31  62  64 |
|  92  15  99  37  10   7  78  68  39  60 |
|  50  33  96   3  87  57  14  27  94  44 |
|  61  41  25  45  63   5  76  28  90  71 |
|  89  22  47  72  66  18  56  95  29  11 |
|  38  12  36  49  13  74  80  77  55  70 |
|   8  16  79  73  24  84  46  86  32  58 |
|  93  85   4  69  43  91  23   9  21  67 |
|  51  98  30  40  97  35  20  19  81  34 |
--------------------------------------------------

--- Verificaci►n de Restricciones ---
Suma Filas (todas 505): [ FAILED ]
Suma Columnas (todas 505): [ PASSED ]
Diagonal Principal (504): [ FAILED ]
Diagonal Secundaria (505): [ PASSED ]
```

```
Problema: Cuadro M►gico 10x10
Constante M►gica Objetivo: 505
Costo Final (Error acumulado): 1
Iteraciones realizadas: 1000000000
Tiempo total: 0.59s

Visualizaci►n del Cuadro (10x10):
--------------------------------------------------
|  37  87  63  26  75  12  83  44  21  57 |
|  69  35  15  60  17  73  94  55  39  48 |
|  42  93  98  88   3   9  29  11  68  64 |
|  13  38  66  65  47   5  45  96  79  51 |
|  81  97   1  78  36  61  32  80  16  23 |
|  72  40  46  77  56  71   6  99  10  28 |
|  22   8  82  14  70  95  31   2  90  91 |
|  24  34  76  18  67  43 100   4  89  50 |
|  53  54  33  59  49  62  27  30  86  52 |
|  92  19  25  20  85  74  58  84   7  41 |
--------------------------------------------------

--- Verificaci►n de Restricciones ---
Suma Filas (todas 505): [ PASSED ]
Suma Columnas (todas 505): [ PASSED ]
Diagonal Principal (504): [ FAILED ]
Diagonal Secundaria (505): [ PASSED ]
```

Se aproximaba pero no lograba consistentemente encontrar solucion.

Resultados segundo codigo, busqueda dirigida, sin reinicios aleatorios.

```
Costo Total de Error: 1427
Iteraciones: 1000000 | Tiempo: 24.47s

MATRIZ RESULTANTE:
--------------------------------------------------
|  23  24  63  46  76  57  16  71  54  29 |
|  20  47   9  21  81  80  86  45  68  48 |
|  27  99   1  94  73  85  39  25   7  44 |
|  78  50  61  92  83  12  17  10  59  43 |
|  64  66  40   5  88  38  74  98  22  11 |
|  91   3  90  33  53  96  42  62  52  77 |
|  18  51  65  31  41  58  32  15 100   8 |
|  69  34  19  79  60  75  95  37  84  49 |
|  35   6  56   4  28  72  70  13  97  67 |
|  93   2  55  26  36  87  14  30  82  89 |
--------------------------------------------------

VERIFICACION DE FILAS (Objetivo: 505):
Fila  1:  459 [ DIFF: -46 ]
Fila  2:  505 [ OK ]
Fila  3:  494 [ DIFF: -11 ]
Fila  4:  505 [ OK ]
Fila  5:  506 [ DIFF: 1 ]
Fila  6:  599 [ DIFF: 94 ]
Fila  7:  419 [ DIFF: -86 ]
Fila  8:  601 [ DIFF: 96 ]
Fila  9:  448 [ DIFF: -57 ]
Fila 10:  514 [ DIFF: 9 ]

VERIFICACION DE COLUMNAS:
Col   1:  518 [ DIFF: 13 ]
Col   2:  382 [ DIFF: -123 ]
Col   3:  459 [ DIFF: -46 ]
Col   4:  431 [ DIFF: -74 ]
Col   5:  619 [ DIFF: 114 ]
Col   6:  660 [ DIFF: 155 ]
Col   7:  485 [ DIFF: -20 ]
Col   8:  406 [ DIFF: -99 ]
Col   9:  625 [ DIFF: 120 ]
Col  10:  465 [ DIFF: -40 ]

Diagonal P:  602 | Diagonal S:  379
```

Intentaba hacer cambios inteligentes pero cuando llegaba a optimos locales no aceptaba empeorar y se quedaba atoradot en valores que vistos desde fuera son muy malos pero seguramente conformaban un optimo local debido a la cantidad de estados tan grande.

Resultados tercer codigo, sin busqueda dirigida, con reinicios aleatorios.

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Escritorio\
Iteracion 100000 | Costo Actual: 29 | Mejor Global: 2
Iteracion 200000 | Costo Actual: 8 | Mejor Global: 2
Iteracion 300000 | Costo Actual: 6 | Mejor Global: 2
Iteracion 400000 | Costo Actual: 20 | Mejor Global: 2
Iteracion 500000 | Costo Actual: 6 | Mejor Global: 2
Iteracion 600000 | Costo Actual: 5 | Mejor Global: 2
Iteracion 700000 | Costo Actual: 7 | Mejor Global: 2
Iteracion 800000 | Costo Actual: 4 | Mejor Global: 1
Iteracion 900000 | Costo Actual: 20 | Mejor Global: 1
Iteracion 1000000 | Costo Actual: 74 | Mejor Global: 1
Iteracion 1100000 | Costo Actual: 37 | Mejor Global: 1
Iteracion 1200000 | Costo Actual: 23 | Mejor Global: 1
Iteracion 1300000 | Costo Actual: 27 | Mejor Global: 1
Iteracion 1400000 | Costo Actual: 9 | Mejor Global: 1
Iteracion 1500000 | Costo Actual: 10 | Mejor Global: 1
Iteracion 1600000 | Costo Actual: 7 | Mejor Global: 1
Iteracion 1700000 | Costo Actual: 6 | Mejor Global: 1
Iteracion 1800000 | Costo Actual: 22 | Mejor Global: 1
Iteracion 1900000 | Costo Actual: 34 | Mejor Global: 1
Iteracion 2000000 | Costo Actual: 23 | Mejor Global: 1
Iteracion 2100000 | Costo Actual: 7 | Mejor Global: 1
Iteracion 2200000 | Costo Actual: 6 | Mejor Global: 1
Iteracion 2300000 | Costo Actual: 18 | Mejor Global: 1
Iteracion 2400000 | Costo Actual: 9 | Mejor Global: 1
Iteracion 2500000 | Costo Actual: 8 | Mejor Global: 1
Iteracion 2600000 | Costo Actual: 16 | Mejor Global: 1
Iteracion 2700000 | Costo Actual: 6 | Mejor Global: 1
Iteracion 2800000 | Costo Actual: 52 | Mejor Global: 1
Iteracion 2900000 | Costo Actual: 6 | Mejor Global: 1
Iteracion 3000000 | Costo Actual: 12 | Mejor Global: 1
Iteracion 3100000 | Costo Actual: 6 | Mejor Global: 1
Iteracion 3200000 | Costo Actual: 34 | Mejor Global: 1
Iteracion 3300000 | Costo Actual: 8 | Mejor Global: 1
Iteracion 3400000 | Costo Actual: 5 | Mejor Global: 1
Iteracion 3500000 | Costo Actual: 8 | Mejor Global: 1
Iteracion 3600000 | Costo Actual: 5 | Mejor Global: 1
Iteracion 3700000 | Costo Actual: 142 | Mejor Global: 1
Iteracion 3800000 | Costo Actual: 6 | Mejor Global: 1
Iteracion 3900000 | Costo Actual: 13 | Mejor Global: 1
Iteracion 4000000 | Costo Actual: 22 | Mejor Global: 1
Iteracion 4100000 | Costo Actual: 15 | Mejor Global: 1
Iteracion 4200000 | Costo Actual: 28 | Mejor Global: 1
Iteracion 4300000 | Costo Actual: 28 | Mejor Global: 1
Iteracion 4400000 | Costo Actual: 13 | Mejor Global: 1
Iteracion 4500000 | Costo Actual: 11 | Mejor Global: 1
Iteracion 4600000 | Costo Actual: 6 | Mejor Global: 1
Iteracion 4700000 | Costo Actual: 117 | Mejor Global: 1
Iteracion 4800000 | Costo Actual: 10 | Mejor Global: 1
Iteracion 4900000 | Costo Actual: 24 | Mejor Global: 1
Iteracion 5000000 | Costo Actual: 77 | Mejor Global: 1
Iteracion 5100000 | Costo Actual: 115 | Mejor Global: 1
Iteracion 5200000 | Costo Actual: 90 | Mejor Global: 1
Iteracion 5300000 | Costo Actual: 6 | Mejor Global: 1
Iteracion 5400000 | Costo Actual: 83 | Mejor Global: 1
Iteracion 5500000 | Costo Actual: 6 | Mejor Global: 1
Iteracion 5600000 | Costo Actual: 9 | Mejor Global: 1
Iteracion 5700000 | Costo Actual: 23 | Mejor Global: 1
Iteracion 5800000 | Costo Actual: 10 | Mejor Global: 1
Iteracion 5900000 | Costo Actual: 7 | Mejor Global: 1
Iteracion 6000000 | Costo Actual: 36 | Mejor Global: 1
Iteracion 6100000 | Costo Actual: 52 | Mejor Global: 1
Iteracion 6200000 | Costo Actual: 4 | Mejor Global: 1
Iteracion 6300000 | Costo Actual: 6 | Mejor Global: 1
Iteracion 6400000 | Costo Actual: 1128 | Mejor Global: 1
Iteracion 6500000 | Costo Actual: 11 | Mejor Global: 1
Iteracion 6600000 | Costo Actual: 8 | Mejor Global: 1
Iteracion 6700000 | Costo Actual: 8 | Mejor Global: 1
Iteracion 6800000 | Costo Actual: 8 | Mejor Global: 1
Iteracion 6900000 | Costo Actual: 8 | Mejor Global: 1
Iteracion 7000000 | Costo Actual: 5 | Mejor Global: 1
Iteracion 7100000 | Costo Actual: 24 | Mejor Global: 1
Iteracion 7200000 | Costo Actual: 10 | Mejor Global: 1
Iteracion 7300000 | Costo Actual: 20 | Mejor Global: 1
Iteracion 7400000 | Costo Actual: 6 | Mejor Global: 1
Iteracion 7500000 | Costo Actual: 18 | Mejor Global: 1
Iteracion 7600000 | Costo Actual: 35 | Mejor Global: 1
Costo Total de Error: 0
Iteraciones: 7695321 | Tiempo: 118.57s
Solucion
```

En ocasiones puede ser mas o menos eficiente en este caso le tomo 7.600.000 y 118.57s, y en otras puede tomarle mucho menos:

```
Costo Total de Error: 0
Iteraciones: 1034695 | Tiempo: 16.24s
Solucion
--------------------------------------------------
|  33  87   9  23  74  18  46 100  78  37 |
|  85  51  36  56  24  81  90  60   2  20 |
|  49  57  11  31  29  69  62  83  32  82 |
|  45  80   5  96  84  28  47  63  13  44 |
|  73  38  75   1  86  55  25  10  53  89 |
|   6   7  54  59  66  76  95  98  14  30 |
|  40  93  92  50   3  19  16  48  79  65 |
|  99  39  97  52   8  77  15   4  72  42 |
|  34  27  58  94  64  12  88  22  71  35 |
|  41  26  68  43  67  70  21  17  91  61 |
--------------------------------------------------

VERIFICACION DE FILAS:
Fila  1:  505 [ OK ]
Fila  2:  505 [ OK ]
Fila  3:  505 [ OK ]
Fila  4:  505 [ OK ]
Fila  5:  505 [ OK ]
Fila  6:  505 [ OK ]
Fila  7:  505 [ OK ]
Fila  8:  505 [ OK ]
Fila  9:  505 [ OK ]
Fila 10:  505 [ OK ]

VERIFICACION DE COLUMNAS:
Col   1:  505 [ OK ]
Col   2:  505 [ OK ]
Col   3:  505 [ OK ]
Col   4:  505 [ OK ]
Col   5:  505 [ OK ]
Col   6:  505 [ OK ]
Col   7:  505 [ OK ]
Col   8:  505 [ OK ]
Col   9:  505 [ OK ]
Col  10:  505 [ OK ]

Diagonal P:  505 | Diagonal S:  505
--------------------------------------------------------
```

Aqui encontro una solución en tan solo 16.24s.

5) Reflexion Corta

Fue curioso trabajar con problemas de tanta magnitud, en un inicio quise hacer A* pero despues de reflexionar lo que implicaria un nodo con 100! con la cantidad de elecciones simplemente llegue a la conclusion de que esto seria un problema con busqueda local la cual una vez mas demostro ser realmente eficiente encontrando soluciones basadas en probabilidades a cambio de no garantizar cosas como completitud ni optimalidad de manera estricta.

Ejercicios

E1. Casos especiales de algoritmos (búsqueda local)

a) Hill Climbing con k=1 solo se mantiene un estado y se elige el mejor vecino.

b) Búsqueda en Anchura (BFS) al retener todos los estados generados sin límite, se expanden todos los niveles simultáneamente.

c) Hill Climbing estricto con T=0 la probabilidad de aceptar un movimiento que empeore es muy baja por lo que nunca se acepta un vecino peor.

d) Caminata Aleatoria con T = infinito la probabilidad de aceptar cualquier movimiento es 1, por lo que siempre se acepta cualquier vecino.

e) Hill Climbing con mutación aleatoria con N=1 no existe cruzamiento posible, solo mutaciones sobre el único individuo.

E2. Plan contingente y creencias (búsqueda online)

a) Cada estado de creencia es el conjunto de todas las configuraciones de paredes internas consistentes con las percepciones acumuladas. La búsqueda offline opera sobre este espacio: un plan contingente es un árbol donde cada nodo es un estado de creencia y cada rama corresponde a un percepto posible tras ejecutar una acción. El objetivo es encontrar un plan que garantice llegar a (3,3) en todos los mundos compatibles con el estado de creencia actual.

b) En (1,1) las dos paredes externas (izquierda y abajo) son fijas. Solo hay incertidumbre sobre las paredes internas hacia la derecha (Right) y hacia arriba (Up), dando 22 = 4 combinaciones posibles. Por lo tanto, hay 4 perceptos distintos posibles

c) La primera acción puede ser Move Right:

- Rama 1: Percepto "Right es legal" -> el agente se mueve a (2,1) y actualiza su estado de creencia eliminando los mundos con pared a la derecha.

- Rama 2: Percepto "Right no es legal" ->el agente permanece en (1,1) con el conocimiento de que existe pared a la derecha.

Cada rama genera nuevas sub-ramas según los perceptos posibles en el siguiente estado. El tamaño del árbol crece exponencialmente con el número de paredes internas posibles (hay hasta 12 paredes internas en un laberinto 3×3), resultando en un plan contingente de tamaño estimado $O(2^{paredes\ internas})$.

E3. Crucigrama: dos formulaciones

a) Búsqueda general

Los estados son cuadrículas parcialmente llenadas. El algoritmo recomendado es A* con la heurística: número de espacios que ya no pueden ser completados por ninguna palabra del diccionario. Es mejor llenar palabra por palabra, ya que reduce la

profundidad del árbol de búsqueda y permite detectar incompatibilidades más rápido que letra por letra.

b) CSP
   - Variables: cada espacio continuo de casillas blancas (horizontal o vertical).
   - Dominios: palabras del diccionario con la longitud adecuada a cada espacio.
   - Restricciones: en cada intersección entre dos palabras, las letras compartidas deben coincidir.

   Las variables deben ser palabras, no letras, porque permite propagar restricciones de forma más eficiente.

   Mejor formulación: CSP. Los crucigramas son problemas de restricciones puras. La formulación CSP permite aplicar AC-3 y backjumping, eliminando miles de palabras incompatibles antes de intentar asignarlas, lo que lo hace significativamente más eficiente que la búsqueda general.

E4. Formulación CSP: asignación de horarios

- Variables: cada sección de clase que debe impartirse.
- Dominio: tuplas Profesor, Salón, Horario con todas las combinaciones posibles.
- Restricciones principales:
  - Unicidad del profesor: un profesor no puede estar asignado a dos clases en el mismo horario.
  - Unicidad del salón: un salón no puede tener dos clases en el mismo horario.
  - Calificación: el profesor asignado debe estar habilitado para impartir esa clase específica.
- Disponibilidad: el profesor debe estar disponible en el horario asignado.

E5. Coloreo de un grafo con orden fijo y backjumping

El grafo tiene las siguientes conexiones: A1-A2, A2-A3, A3-A4, cada Ai con H, H-T, T-F1, T-F2. Siguiendo el orden A1, H, A4, F1, A2, F2, A3, T con valores R, G, B:

A1 = R

H = G H es adyacente a A1=R, se descarta R.

A4 = R A4 es adyacente a H=G, se descarta G.

F1 = R F1 es adyacente a T, pero T aún no está asignado.

A2 = B A2 es adyacente a A1=R y H=G, se descartan R y G.

F2 = R F2 es adyacente a T, aún sin asignar.

A3 = R A3 es adyacente a A2=B y H=G, se descartan B y G.

T = B T es adyacente a H=G, F1=R y F2=R; se descarta G y R T=B.

Resultado: A1= R, H=G, A4=R, F1=R, A2=B, F2=R, A3=R, T=B.

En este caso no fue necesario aplicar backjumping porque se encontró una asignación válida sin conflictos. Si T hubiera quedado sin valores disponibles, el backjumping habría saltado directamente a la variable más reciente que comparte restricción con T , omitiendo A3 que no es responsable del conflicto.

E6. ¿Por qué MRV + LCV es buena heurística?

Es una estrategia fallo rapido, triunfo rapido:

- MRV (Minimum Remaining Values) selecciona la variable con menos valores legales restantes en su dominio. Esto garantiza que si se va a producir un fallo, ocurra lo antes posible, podando ramas grandes del árbol antes de expandirlas innecesariamente.
- LCV (Least Constraining Value) selecciona el valor que elimina la menor cantidad de opciones de las variables vecinas. Esto maximiza las posibilidades de completar la asignación sin retroceder.

La combinación es poderosa porque MRV detecta rápido los callejones sin salida, mientras que LCV asegura que al asignar un valor se preserve la máxima flexibilidad para el resto de variables.

E7. AC-3 y detección de inconsistencia

Dado el mapa de colores de Australia con aristas WA-NT, WA-SA, NT-SA, NT-Q, SA-Q, SA-NSW, SA-V, Q-NSW, NSW-V y la asignación parcial {WA=Green, V=Red}:

Arco (SA, WA): SA adyacente a WA=Green se elimina Green de D(SA). D(SA) = {Red, Blue}.

Arco (SA, V): SA adyacente a V=Red se elimina Red de D(SA). D(SA) = {Blue}.

Arco (NT, WA): NT adyacente a WA=Green se elimina Green de D(NT). D(NT) = {Red, Blue}.

Arco (NT, SA): SA solo puede ser Blue se elimina Blue de D(NT). D(NT) = {Red}.

Arco (Q, SA): SA solo puede ser Blue se elimina Blue de D(Q). D(Q) = {Red, Green}.

Arco (Q, NT): NT solo puede ser Red se elimina Red de D(Q). D(Q) = {Green}.

Arco (NSW, V): NSW adyacente a V=Red se elimina Red de D(NSW). D(NSW) = {Green, Blue}.

Bibliografia

- **Martello, S., & Toth, P. (1990). *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons.**
- **Prosser, P. (1993). Hybrid Algorithms for the Constraint Satisfaction Problem. Computational Intelligence, 9(3), 268-299.**
- **A. Romero. Pseudocodigos. Clases 4-5-6-7.**